## INTERNSHIP PROJECT REPORT

# Spam Email Classifier

Machine Learning based Email Classification using TF-IDF and Multinomial Naive Bayes


| | |
|---|---|
| **Student Name** | Aman Kumar |
| **Roll Number** | 25scs1003003295 |
| **Course / Branch** | Btech CSE |
| **Organization / College** | IILM University |
| **Submission Date** | 28 August 2026 |


*Prepared for internship project evaluation*

## Abstract

Spam emails are unwanted messages that can waste time, carry misleading offers, or attempt to collect sensitive information. This project develops a small, reproducible machine-learning system that classifies an email message as spam or ham (not spam). The solution uses TF-IDF to convert email text into numerical features and a Multinomial Naive Bayes classifier to make the prediction. A lightweight offline dataset of 1,200 labeled demonstration emails is included so the project can be trained and executed without an internet connection. On a stratified 80/20 train-test split, the implemented model achieved 100% accuracy on the included demonstration dataset. The project also provides an interactive command-line application and a batch prediction script.

## 1. Introduction

Email remains a common communication channel, but users receive a large number of unwanted messages. Manual filtering is inconvenient, while keyword-only rules can be brittle because spammers can change wording. Text classification offers a simple data-driven approach: learn patterns from labeled messages and use those patterns to predict the class of a new message.

This project is intentionally designed at an introductory internship level. It demonstrates the complete machine-learning pipeline from data preparation through model training, evaluation, model persistence, and prediction.

## 2. Problem Statement

Develop a machine-learning based system that reads the textual content of an email and predicts whether the message is spam or ham. The system should be simple to train, easy to run locally, and suitable for demonstrating fundamental NLP and classification concepts.

## 3. Objectives

- Prepare a labeled dataset of spam and ham email text.
- Convert text into numerical features using TF-IDF.
- Train a Multinomial Naive Bayes classifier.
- Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Save the trained model so predictions can be made without retraining.
- Provide a simple interactive command-line demonstration and batch prediction utility.

## 4. Technology Stack

| Technology | Purpose | Why Used |
|---|---|---|
| Python | Core implementation | Simple syntax and strong ML ecosystem |
| Pandas | Dataset handling | Easy CSV loading and manipulation |
| Scikit-learn | NLP + machine learning | TF-IDF, Naive Bayes, train/test split, metrics |
| Joblib | Model storage | Save and reload the trained pipeline |

## 5. System Architecture

The system follows a straightforward supervised-learning pipeline:

- Email dataset -> labeled spam/ham text
- Train/test split -> 80% training and 20% testing
- TF-IDF vectorization -> convert text into weighted token features
- Multinomial Naive Bayes -> learn class probabilities from training data
- Evaluation -> calculate classification metrics on unseen test messages
- Saved pipeline -> load the trained model for new email predictions

## 6. Dataset

The repository contains 1,200 labeled demonstration emails: 600 spam and 600 ham. The data is stored locally in data/emails.csv with two columns: label and text. The examples were generated from varied templates to keep the project self-contained and deterministic for offline demonstration.

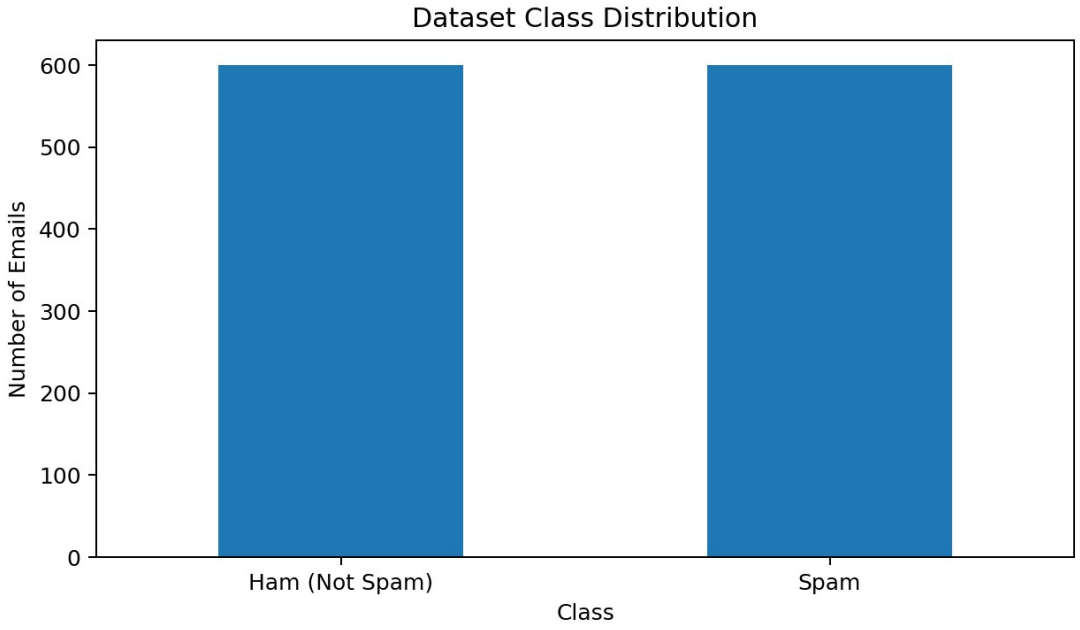


Figure 1. Class distribution of the bundled demonstration dataset.

## 7. Text Preprocessing and Feature Extraction

Machine-learning algorithms need numerical input. Raw email text is therefore transformed with TF-IDF (Term Frequency-Inverse Document Frequency). The vectorizer lowercases the text, removes English stop words, and creates unigram and bigram features. Sublinear term frequency is enabled so repeated words do not grow linearly without bound.

### TF-IDF concept

For a term t in document d, TF-IDF can be represented conceptually as:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

Terms that are important within a message but not common across the entire dataset receive larger weights. This gives the classifier useful signals without requiring a hand-built keyword list.

## 8. Machine Learning Algorithm

The classifier is Multinomial Naive Bayes. It is a common baseline for document classification because it is fast, simple, and works well with non-negative word-count or TF-IDF style features. The model learns how strongly the observed terms are associated with each class and combines those probabilities to produce a final label.

The implementation uses a scikit-learn Pipeline so the TF-IDF transformation and classifier remain together. The same feature mapping is automatically reused when a new email is predicted.

## 9. Training and Evaluation

The dataset is split into 80% training data and 20% test data using stratification and a fixed random seed (42). Stratification preserves the spam/ham class ratio in both subsets. The test set is kept separate from training so the reported metrics reflect performance on unseen examples.

| Metric | Score |
|---|---|
| Accuracy | 100.00% |
| Precision (spam) | 100.00% |
| Recall (spam) | 100.00% |
| F1-score (spam) | 100.00% |

**Confusion matrix: [[120, 0], [0, 120]] for [ham, spam].**

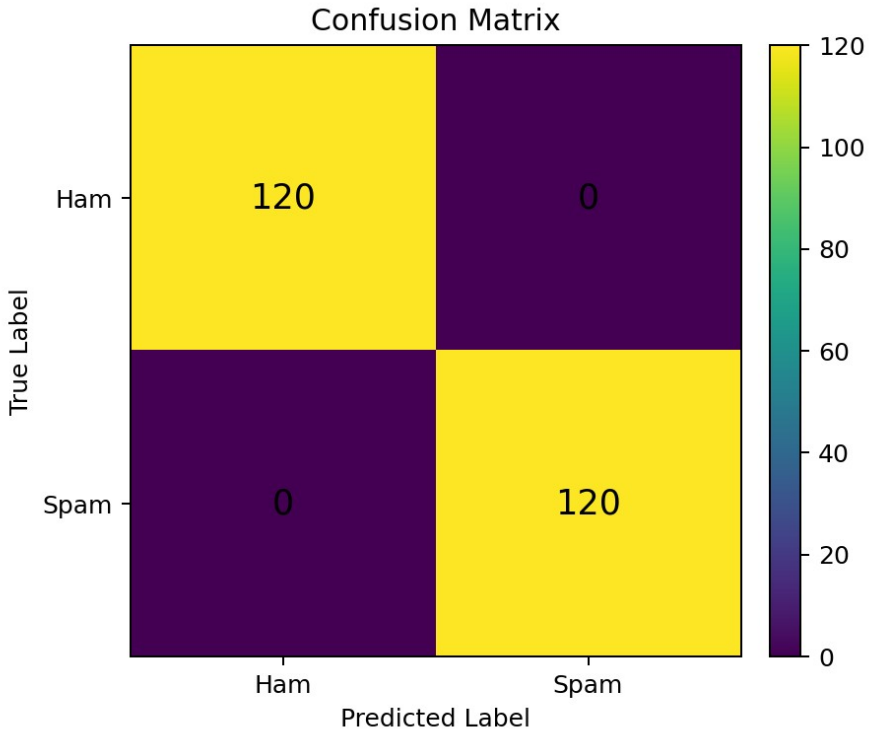


Figure 2. Confusion matrix on the 20% test set.

The 100% score should not be interpreted as proof that the model will be perfect on real-world email. The bundled dataset is small and template-based, so the result mainly demonstrates that the complete implementation works. A production classifier should be evaluated on a large, independently collected corpus and monitored for false positives and changing spam patterns.

## 10. Application and Usage

The project includes an interactive command-line application. After installing the requirements and running train_model.py, execute app.py and enter an email message. The application prints the predicted class and the model confidence for that prediction.

```
Train: python train_model.py
Run demo: python app.py
Batch mode: python batch_predict.py data/demo_emails.csv -o predictions.csv
```

## 11. Sample Predictions

| Input message | Expected class | Observed demo behavior |
|---|---|---|
| Congratulations! You won a free cash prize. Click here to claim it now. | Spam | Spam |
| Hi, please share the meeting notes when you get a chance. | Ham | Ham |
| Urgent: verify your account now to avoid suspension. | Spam | Spam |
| The assignment is due tomorrow. Please submit it through the portal. | Ham | Ham |

## 12. Advantages

- Simple and fast to train.
- Works directly with email text.
- Easy to reproduce offline.
- Pipeline stores both preprocessing and classification logic.
- Provides standard classification metrics for evaluation.

## 13. Limitations

- The included dataset is a demonstration dataset, not a production-grade email corpus.
- The model may fail when spam is written in unusual or obfuscated language.
- Email metadata, sender reputation, URLs, attachments, and headers are not used.
- False positives can be costly and require careful threshold and dataset management in real deployments.
- The 100% demo accuracy is specific to this bundled dataset and should not be generalized.

## 14. Future Scope

- Train on a larger public or organization-approved corpus.
- Add email headers, URL statistics, sender reputation, and attachment metadata.
- Compare Naive Bayes with Logistic Regression, Linear SVM, or transformer-based models.
- Expose the classifier through a REST API or web dashboard.
- Add periodic retraining and drift monitoring as spam patterns change.

## 15. Conclusion

The Spam Email Classifier demonstrates the core workflow of an NLP classification project: data preparation, feature extraction, supervised learning, evaluation, and deployment of a saved model. The

implementation is intentionally compact, understandable, and runnable on a normal Python installation. It is suitable for demonstrating introductory machine-learning skills during an internship evaluation while leaving clear paths for further improvement.

## 16. References


- Scikit-learn documentation: TfidfVectorizer.
- Scikit-learn documentation: MultinomialNB.
- Scikit-learn documentation: model evaluation metrics and Pipeline.
- Pandas documentation: CSV data handling.